



UNIVERSITÀ DI PISA

REPORT DATA MINING: FUNDAMENTALS

Exploration IMDb Dataset

Clarissa Pia Rizzello

620329

Lavinia Rotellini

581553

Rachele Sabbatini

660188

Contents

1	Introduction	2
2	Data Understanding and Transformation	2
2.1	Data Semantics	2
2.2	Distribution and Statistics	3
2.3	Data Quality and Correlations	5
2.3.1	Handling Missing Values and Duplicates	5
2.4	Data Transformation	5
2.5	Outliers	6
2.6	Pairwise Correlations and Elimination of Variables	7
3	Clustering	8
3.1	Preprocessing towards clustering	9
3.2	Centroid-based Clustering: KMeans	9
3.3	Density-based Clustering	10
3.3.1	DBScan	10
3.3.2	HDBScan	10
3.4	Agglomerative Clustering	11
3.5	Discussion of the results	11
4	Classification	12
4.1	Data preparation	13
4.2	KNN	13
4.3	Naïve Bayes	14
4.4	Decision Tree	14
4.5	Classification conclusion	16
5	Regression	16
5.1	Results of the Multivariate Regression with One Target Variable	16
5.2	Feature Importances for the Linear Regressor	17
5.3	Results of the Multivariate Regression with Two Target Variables	17
6	Pattern Mining	17
6.1	Data Preparation	17
6.2	Frequent Pattern Extraction	18
6.2.1	Discussion on Frequent Patterns	18
6.3	Association Rules Extraction	19
6.3.1	Discussion on Association Rules	20
6.4	Predictive Use of the Rules	22
6.5	Conclusions	23

1 Introduction

In this study, we conducted a data mining analysis on an IMDb dataset. We began with the data understanding phase, exploring and preparing the dataset's attributes to gain deeper insights. Subsequently, we applied various clustering algorithms, including K-Means, DBSCAN, and Hierarchical Clustering. Following clustering, we progressed to classification, training multiple models such as K-Nearest Neighbors (KNN), Naïve Bayes, and Decision Trees. Our exploration also extended to regression analysis, and finally, we employed association rule mining techniques, specifically Apriori and FP-Growth algorithms, to identify meaningful patterns within the data.

2 Data Understanding and Transformation

This dataset from IMDb contains information about movies and TV shows, including key attributes such as titles, release years, runtimes, user ratings, awards, and genres.

A "soft" data preparation strategy was adopted, as the objective was to create a consistent and coherent baseline to be used across various required tasks (clustering, classification, pattern mining, regression), each of which might necessitate further specific transformations. In this preliminary phase, we handled missing values, identified and corrected outliers, removed irrelevant or redundant features, and conducted general data cleaning operations. All modifications were documented and saved in a new `.csv` file, serving as the shared starting point for subsequent analyses.

2.1 Data Semantics

Our dataset was composed of 16430 records with 23 descriptive features. The features were both categorical (*qualitative*) and numeric (*quantitative*). Table 1 summarizes the key features of the dataset.

Table 1: Feature descriptions and types

Feature	Feature Type	Description
originalTitle	Categorical, Nominal	Original title of the work, unique identifier expressed in the original language.
runtimeMinutes	Numeric, Integer	Primary runtime of the title, expressed in minutes.
isAdult	Categorical, Boolean	Binary indicator specifying whether the content is adult (1) or not (0).
startYear	Numeric, Discrete	Release year of the title; for TV series, the start year of the series.
endYear	Numeric, Discrete	TV Series end year, if applicable.
numVotes	Numeric, Discrete	Total number of votes received by the title.
numRegions	Numeric, Discrete	Number of geographic regions where this version of the title is available.
worstRating	Numeric	Lowest rating value assigned to the title, minimum on the rating scale.
bestRating	Numeric	Highest rating value assigned to the title, maximum on the rating scale.
canHaveEpisodes	Categorical, Boolean	Indicates whether the title can include episodes.
isRatable	Categorical, Boolean	Indicates whether the title can be rated by users.

Continued on next page

Feature	Feature Type	Description
totalImages	Numeric, Discrete	Total number of images associated with the title's IMDb page.
totalVideos	Numeric, Discrete	Total number of videos associated with the title's IMDb page.
totalCredits	Numeric, Discrete	Total number of credits associated with the title.
criticReviewsTotal	Numeric, Discrete	Total number of professional critic reviews.
awardWins	Numeric, Discrete	Total number of awards won by the title.
awardNominationsExcludeWins	Numeric, Discrete	Total number of award nominations excluding wins.
titleType	Categorical, Nominal	Category or format of the title (e.g., movie, short, TV series, episode, video).
rating	Numeric, Interval	Average IMDb rating of the title.
ratingCount	Numeric, Discrete	Total number of user ratings submitted for the title.
countryOfOrigin	Categorical, Multi-valued Nominal	Country where the title was primarily produced.
genres	Categorical, Multi-valued Nominal	Genres associated with the title.
userReviewsTotal	Numeric, Discrete	Total number of user-written reviews.

2.2 Distribution and Statistics

In this section, the most important features used in our subsequent analysis are presented, along with other interesting findings.

rating

The **rating** variable represents an average evaluation score for the titles. Originally expressed as intervals (e.g., (7, 8]), it was transformed into a numerical value by averaging the two numbers (e.g., 7.5). In image 1 the distribution of each class. As it can be seen, 7.5 is the most populated class (4,822 titles), followed by 6.5. Also, Extreme classes 0.5 and 1.5 contain only 67 titles, confirming the low frequency of very low ratings.

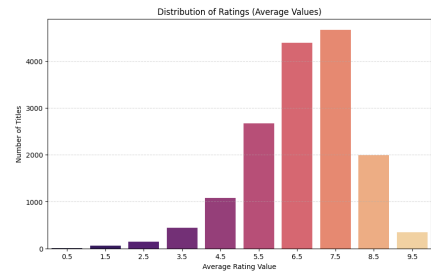


Figure 1: Distribution of Average Rating Classes

titleType

This categorical feature indicates the type of the title. The bar chart in Figure 2 shows a clear dominance of types such as `movie`, followed by `tvSeries` and `short`. Numerous aggregated metrics were analyzed for each category in order to highlight structural differences and support further feature selection or segmentation.

One of the most relevant graphs, Figure 3, shows the average duration of works for each `titleType`. A high degree of variability between categories is evident:

- `tvMiniSeries` have the highest average duration, followed by `movie` and `tvSpecial`.
- Formats such as `short` and `tvShort` naturally have shorter durations (around 10–15 minutes).
- Error bars indicate notable variance, especially within `tvMiniSeries`, highlighting strong internal heterogeneity.

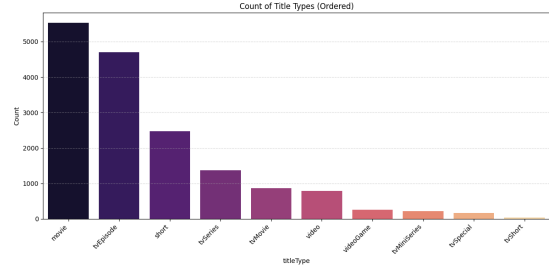


Figure 2: Distribution of Title Types

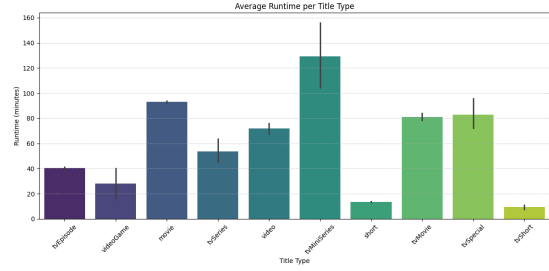


Figure 3: Average Duration by Title Type

Other significant findings:

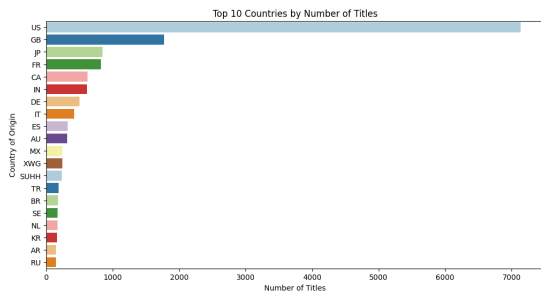
- `userReviewsTotal`: `movie` and `tvSeries` receive far more user comments compared to other types.
- `numRegions`: `movie` titles are distributed across more regions on average, suggesting greater dissemination and international coverage.
- `criticReviewsTotal`: `movie` titles also receive significantly more attention from critics.
- `awardNominationsExcludeWins` and `awardWins`: `tvSeries` and `movie` are the categories with the highest number of recognitions, indicating higher perceived quality or relevance.

The `videoGame` category was removed because it is not consistent with the cinematic or television nature of the other categories. Its inclusion could have introduced semantic noise into the models.

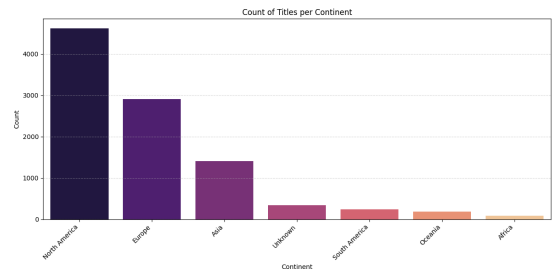
countryOfOrigin

A statistical summary of this feature revealed the diversity of countries represented in the dataset: as shown in Figure 4a, the distribution is highly skewed, with The United States (US) dominating with over 6,500 titles, followed by the United Kingdom (GB) and Japan (JP). Other countries have significantly lower representation.

To reduce the cardinality and asymmetry of the `countryOfOrigin` feature, a new variable `continent` was introduced via ISO mapping using the `pycountry` library. As shown in Figure 4b, the transformation into continents helps aggregate sparse countries into broader and more interpretable categories.



(a) Distribution of Titles by Country of Origin



(b) Distribution of Titles by Continent(after exploding multi-continent entries)

2.3 Data Quality and Correlations

2.3.1 Handling Missing Values and Duplicates

From the initial exploration phases, a significant presence of missing values in various variables emerged. Although NaN values were initially explicitly present only in the `awardWins` column (2,618 values), the string `\N` was found to recur in multiple fields. After converting this string to NaN, the following missing value percentages were identified:

- `endYear`: **95.05%**
- `runtimeMinutes`: **29.53%**
- `awardWins`: **15.93%**
- `genres`: **2.32%**

The absence of duplicates was also verified, both absolutely and with respect to the `originalTitle` column, which proved to be unique.

The following strategies were adopted for handling missing values:

- `endYear`: removed due to the excessive percentage of missing data (~95%), compromising its utility.
- `genres`: rows with missing values were deleted. Reliable imputation techniques capable of credibly assigning a genre were not deemed feasible.
- `awardWins`: missing values were replaced with **0**, assuming that the absence of data corresponds to no recorded awards. This choice was supported by the observed distribution, in which 0 predominates.
- `runtimeMinutes`: rows with missing values were filled using the average runtime value filtered by `titleType`.

2.4 Data Transformation

Further preprocessing was done on the following features:

- `isRatable` showed no deviation in the distribution, therefore it was dropped.
- the `isAdult` variable was also dropped, since its information was already present in the `genres` column.
- Entries that showed a `startYear` prior to 1885 were dropped, since canonically cinematic production is set to start with the first movie made by the Lumière Brothers. This variable was also condensed into a categorical and interval new variable called **decade**, since it originally showed 133 different values, which was too granular of an amount for the tasks considered.
- the information contained in the `userReviewsTotal` and `criticsReviewsTotal` was condensed into a new feature: `totalReviews`, while also retaining the original variables.
- `runtimeMinutes` was also transformed in a categorical feature, dividing the original features into short, medium and long length. However, the original feature was kept to be used for clustering.
- the `numVotes` and `ratingCount` were found to be extremely correlated, as will be discussed in a later section, therefore they were condensed into a new numeric integer variable called `num_ratings`

2.5 Outliers

During the data cleaning phase, a systematic analysis of outliers was conducted for several continuous numerical features.

Initially, records with implausible durations were removed:

$$\text{runtimeMinutes} \leq 0 \quad \text{or} \quad \text{runtimeMinutes} > 300$$

A custom Python class was then implemented to detect and remove outliers, with two available methods:

- **Interquartile Range (IQR)**: uses quartiles to identify extreme values.
- **Percentiles**: removes data below the 0.1th percentile and above the 99.9th percentile.

In our case, the method based on *extreme percentiles* was chosen, proving particularly effective for highly skewed and unbalanced distributions, as observed in the analyzed variables. Using IQR would have resulted in the removal of approximately one-third of the dataset.

Variables Considered The columns analyzed for outlier detection were: `numVotes`, `userReviewsTotal`, `num_ratings`, `awardNominationsExcludeWins`, `awardWins`, `criticReviewsTotal`, `runtimeMinutes`.

For each variable, the acceptable interval was calculated as:

$$[x_{\min}, x_{\max}] = [P_{0.1}, P_{99.9}]$$

Where $P_{0.1}$ and $P_{99.9}$ represent the 0.1st and 99.9th percentiles, respectively. Values outside this range were considered outliers and removed. Figure 5 shows *before vs. after* graphs, with thresholds indicated by dashed lines.

A total of 71 outliers were removed, equivalent to approximately **0.43%** of the original dataset. The impact was visually confirmed through the plots below.

- Extreme values in `userReviewsTotal`, `num_ratings`, and `criticReviewsTotal` caused strong distribution skewness.
- For `runtimeMinutes`, a **group-based** removal logic was applied, using `titleType` as an aggregation criterion. This accounted for the significant heterogeneity among different formats (e.g., *short*, *movie*, *tvMiniSeries*).

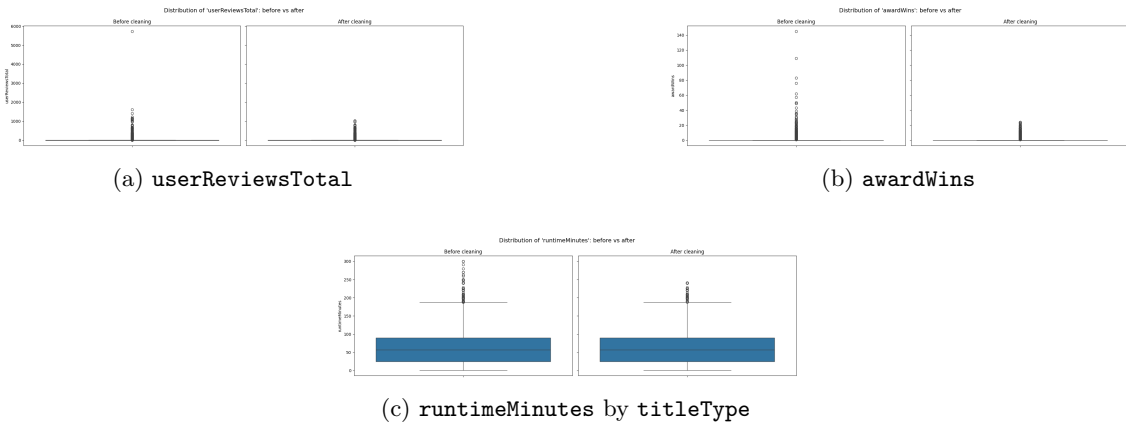


Figure 5: Distribution of numerical variables before and after outlier removal (extreme percentiles)

In order not to excessively alter the original data distribution, we deliberately chose to retain a limited number of moderate outliers. This allowed us to preserve the dataset's heterogeneity while achieving a reasonable trade-off between robustness and representativeness.

2.6 Pairwise Correlations and Elimination of Variables

To identify redundancy and strengthen the quality of the feature set, a correlation analysis was carried out on the numerical variables retained after preprocessing. The resulting Pearson correlation matrix is shown in Figure 6.

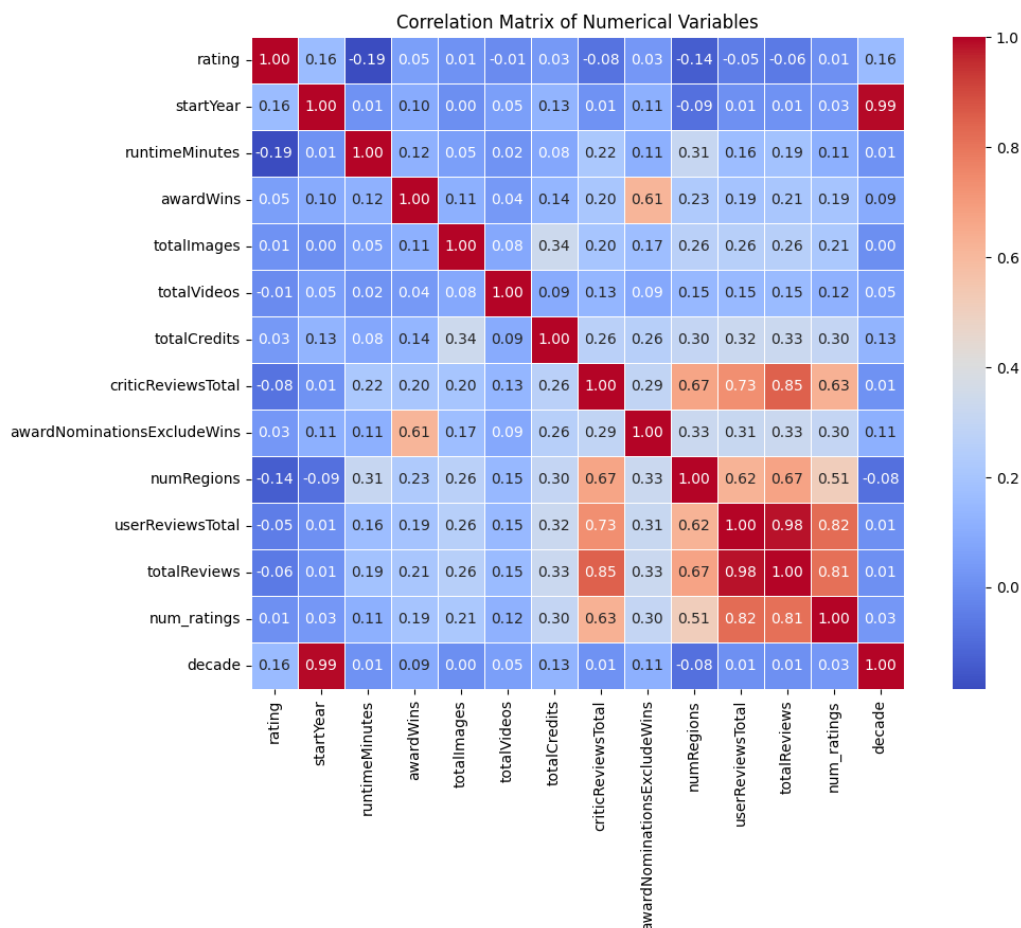


Figure 6: Correlation matrix of numerical features

The matrix highlights some strong and expected associations: notably, **awardWins** correlates positively with **awardNominationsExcludeWins**, and both **userReviewsTotal** and **totalReviews** are closely linked to **criticReviewsTotal** and **numVotes**. These relationships suggest that user engagement, critic presence, and review volume are jointly informative dimensions of title popularity.

Moderate correlations are also present between production-oriented variables such as **totalCredits**, **totalImages**, and **numRegions**, although no excessive collinearity was observed.

A noteworthy preprocessing step was the consolidation of the highly correlated variables **numVotes** and **ratingCount** (previously showing $r > 0.98$), which were replaced by a single aggregated feature named **num_ratings**. This transformation retained their informational content while eliminating redundancy.

Additionally, two columns—**worstRating** and **bestRating**—were removed from the dataset due to their negligible variance. Both had almost constant values and therefore lacked any meaningful contribution to model discrimination.

Pairwise Relationships with userReviewsTotal To further examine the features most associated with audience participation, we plotted the pairwise relationships between **userReviewsTotal** and its most correlated variables. The resulting graph is shown in Figure 7.

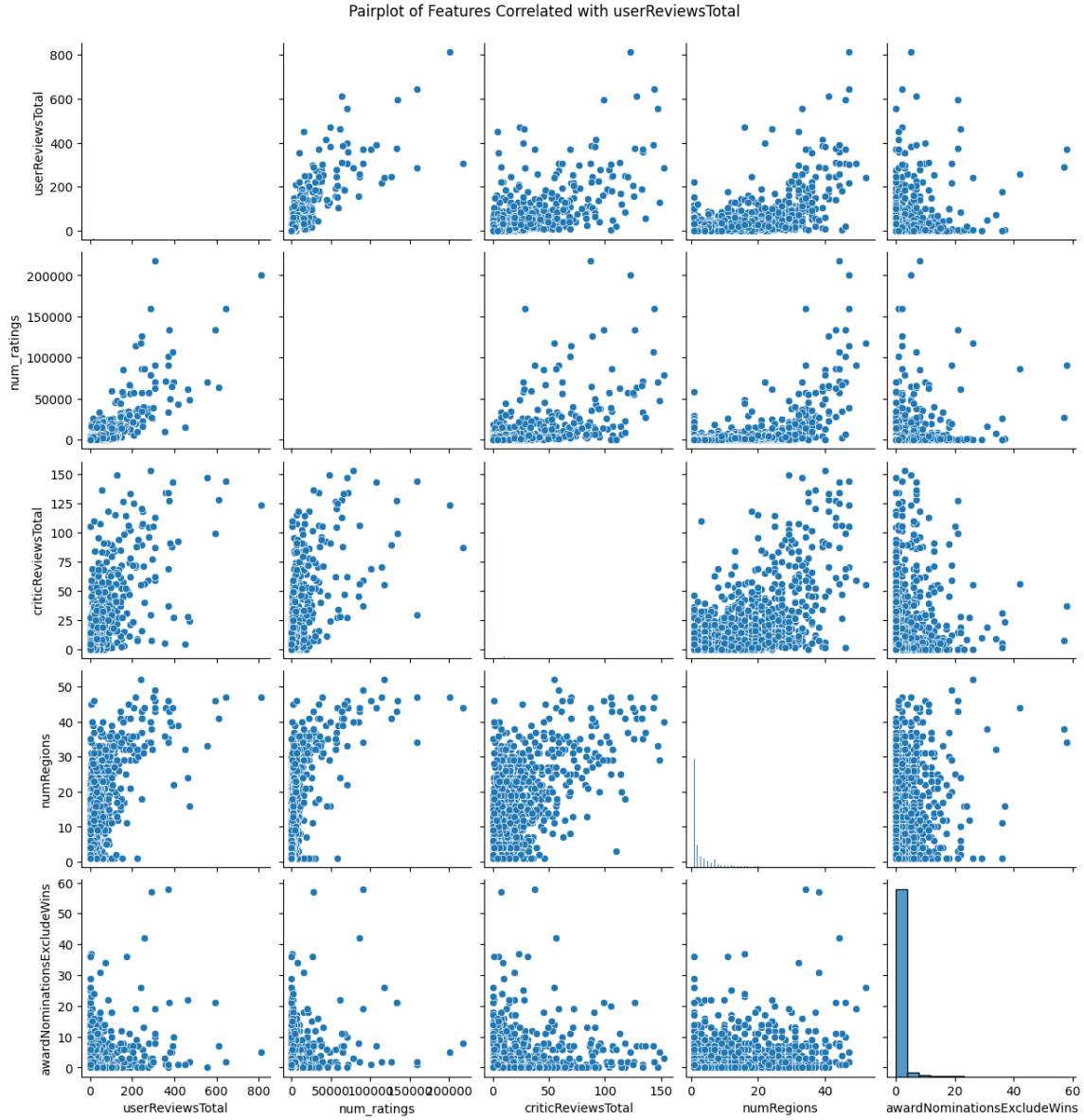


Figure 7: Selected pairwise correlations with `userReviewsTotal`

From the plot, `userReviewsTotal` shows a clear upward trend in relation to `num_ratings` and `criticReviewsTotal`, confirming their role as key indicators of content visibility and interest. Some clustering is visible at low values of `awardNominationsExcludeWins`, with sparse but distinct increases in user reviews among titles with higher nomination counts. The dispersion of values in `numRegions` also reflects the broader distribution reach of certain titles, especially those with global or multi-market releases.

3 Clustering

In this section we provide the results of the clustering analysis, which was performed using the following algorithms:

- KMeans, from the family of instance-base clustering algorithms
- DBScan and HDBScan, from the family of density-based clustering algorithms
- Agglomerative (or hierarchical) Clustering. This last method was applied both with and without the connectivity constraint, and in all of the different versions: Single Link, Average Link, Complete

Link and Ward.

3.1 Preprocessing towards clustering

The dataset used was the one resulting from the data understanding phase, but with further pre-processing specific to the task of clustering.

As a preliminary step columns of 'object' and 'boolean' types were removed: these columns were `originalTitle`, `titleType`, `canHaveEpisodes`, `countryOfOrigin` and `genres`. It was also decided to drop the `decade` column which, although numerical, is considered categorical and therefore not useful in an analysis based on distances. Other columns dropped were `userReviewsTotal` and `criticReviewsTotal`, since their information was previously condensed into `totalReviews`. The dataset was therefore now composed of the following features: `rating`, `startYear`, `runtimeMinutes`, `awardWins`, `totalImages`, `totalVideos`, `totalCredits`, `awardNominationExcludeWins`, `numberOfRegions`, `totalReviews`, `num_ratings`.

The main focus of this analysis was to understand how different types of preprocessing affected the performances of the different algorithms, and which ones were the most appropriate for our data. Therefore, the features transformation was performed in four different ways:

- The first transformation was performed using only the MinMax scaling method
- The second one used both the MinMax Scaling and the logarithmic transformation
- The third transformation only entailed Standard Scaling
- The fourth one used Standard Scaling and the logarithmic transformation

3.2 Centroid-based Clustering: KMeans

The first step towards the application of KMeans was to understand the most appropriate amount of clusters for our dataset: the **Elbow Method** was applied, finding that different types of preprocessing required different ks, as it can be seen in table 2:

Preprocessing	K
MinMax	4
MinMax+Log	4
Standard Scaling	5
Standard+Log	3

Table 2: KMeans Results - Elbow Method

The clustering results were suboptimal and differed between most methods. In table 3, the different SSEs and silhouette scores.

Preprocessing Method	SSE	Silhouette Score
MinMax	629.92	0.30
MinMax & Log	1723.69	0.17
Standard Scaling	74678.86	0.24
Standard & Log	85385.59	0.24

Table 3: KMeans Clustering Results

The results are a clear sign that our dataset may lack an hidden natural structure, which would make it unfit for clustering.

3.3 Density-based Clustering

3.3.1 DBScan

To find the best epsilon for the DBScan algorithm, a list of values was initialized and looped over, with the goal of understanding how many clusters and which silhouette value each epsilon would generate. The *min_samples* parameter was set to 3. In table 4 the results.

Preprocessing	ϵ	Silhouette Score
MinMax	0.3	0.57
MinMax & Log	0.3	0.15
Standard Scaling	0.1	-0.38
Standard & Log	0.1	0.92

Table 4: DBScan Clustering Results

Just by looking at the silhouettes, DBScan seems to be the best method so far, but a further visual analysis showed the clear limitations of this approach: in all the cases DBScan found one cluster containing nearly every instance, and a few others containing very small amounts of records. Furthermore, the identified clusters exhibit substantial overlap, showing that DBScan does not apply properly to our data.

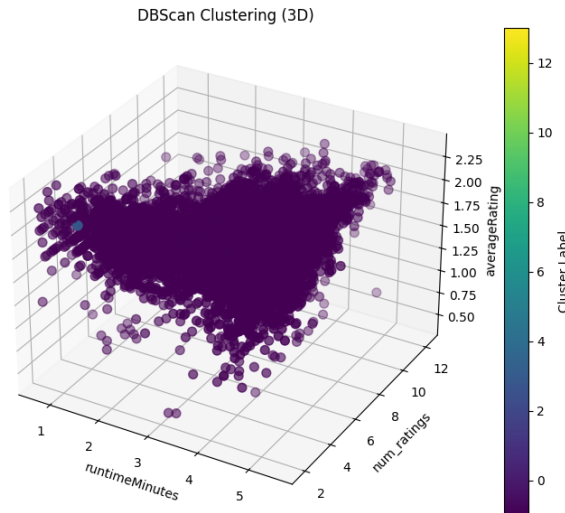


Figure 8: An example of the DBScan application on the dataset with standard scaling and logarithmic transformation.

3.3.2 HDBScan

Since DBScan performed so poorly, HDBScan was also applied to understand whether a more complex density-based algorithm was a better choice. The results were not surprising, since it is now clear that our data does not show varying density and therefore is not of easy analysis by algorithms of this family.

Preprocessing	Silhouette Score
MinMax	-0.16
MinMax & Log	-0.05
Standard Scaling	-0.17
Standard & Log	0.05

Table 5: HDBScan Clustering Results

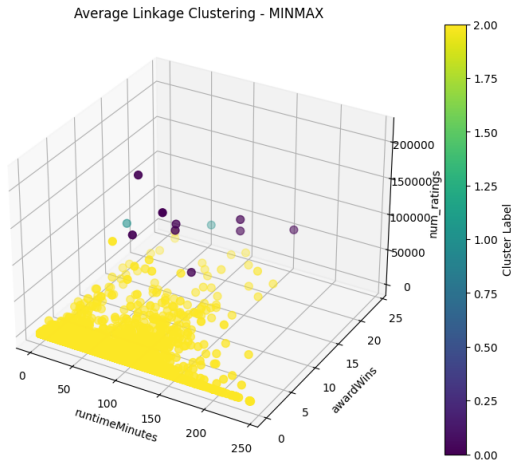
3.4 Agglomerative Clustering

The analysis started without applying the connectivity constraint, but after further inspection it was added as it provided better results. A search over a range of 2 to 10 numbers of clusters was also performed, to understand how the silhouette scores changed based on the hyper-parameter. It was found that after a total of three clusters there was a significant drop in the quality, from around 0.60 to around 0.10 for all the algorithms. In table 6, the best results for each type of preprocessing.

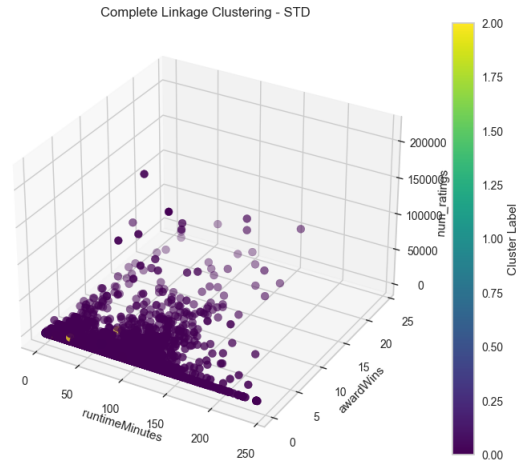
Preprocessing	Algorithm	Silhouette Score
MinMax	Average Link	0.70
MinMax & Log	Average Link	0.50
Standard Scaling	Complete Linkage	0.92
Standard & Log	Average Linkage	0.71

Table 6: Hierarchical Clustering Results

Although the results of the silhouette scores seemed promising, a further inspection revealed that the clusters found by all the different hierarchical clustering algorithms do not reflect interesting meaning in the data. As it is highlighted in the example images below, all of the algorithms have found one single huge cluster, and a few others with little to no instances in them.



(a) Average Linkage Clustering Results on the Min-Max Scaled Dataset



(b) Complete Linkage Clustering Results on the Standard Scaled Dataset

3.5 Discussion of the results

The best partition found was certainly the one provided by KMeans, although its silhouette score was not the highest, but as was discussed in previous sections a high silhouette score never resulted in a meaningful clustering. We will proceed now with a deeper analysis of the clustering resulting from KMeans on the MinMax scaled dataset. Firstly, we tried to understand the usage of the different features by KMeans, which is visualized in image 10.

As it can be seen, KMeans tends to prefer certain features: it is important to notice that the less used features are the ones that bare the least amount of information. More specifically, `awardWins`, `totalImages`, `totalVideos` and `totalCredits` are very inconsistent, the most present value being 0 with just a few peaks. However, the semantics extracted from the features do not appear to be reflected in the resulting clusters, which can be seen in image 11. The visualization uses three key features: `runtimeMinutes`, `awardWins` and `startYear`.

The clusters are all very close and there is no clear separation between them. Nonetheless, an attempt can be made to infer their potential semantic distinctions:

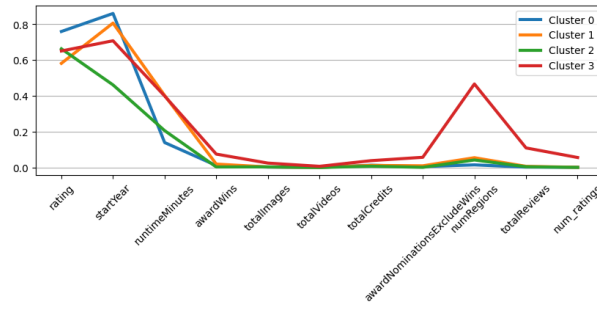


Figure 10: Usage of different features by KMeans

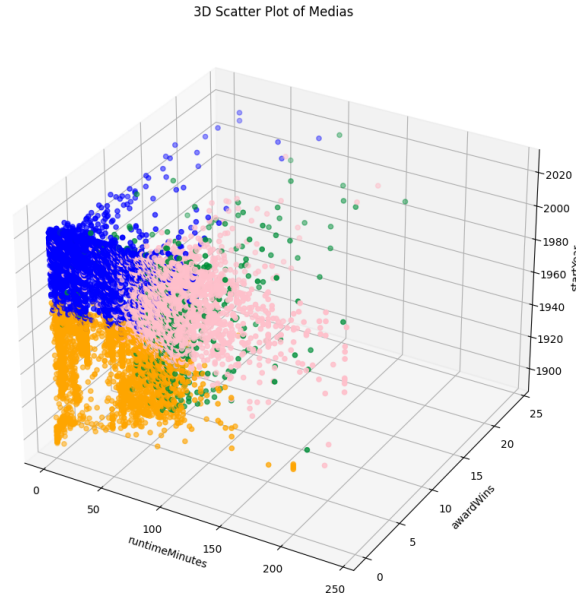


Figure 11: 3d Scatter Plot of the KMeans clusters

- The orange cluster contains products of heterogenous length and little to no `awardWins`, but it very clearly identifies older productions. It can be hypothesized that this group refers to forgotten media which did not pass the test of time, since a cross-analysis with the number of ratings also showed the same cluster in the same position, therefore making it clear that these products are not of particular interest nowadays.
- The green cluster identifies yet again an heterogenous group: for the most part older products, with a strong prevalence of shorter media but also a few longer ones, and an higher count of `awardWins`. This group may refer to old media which are the most critically acclaimed.
- The blue cluster is mainly composed of newer products of shorter length and a varying amount of `awardWins`, we could therefore interpret it as containing mostly newer television and shorter production of varying success.
- The pink cluster is composed of products with longer duration, which probably identifies films, varying number of ratings and mostly newer products. The media in this cluster are the newer movies and longer productions, containing both the ones that received a few awards and more critically acclaimed one.

4 Classification

We used supervised learning to predict categorical outcomes, specifically the type of media product (`titleType`). Three algorithms were evaluated: K-Nearest Neighbors (KNN), Naive Bayes, and Decision

Tree. Each model was trained on a labeled dataset and tested on a separate set.

4.1 Data preparation

To ensure consistency and comparability across classifiers, we used a common dataset for all models. However, specific preprocessing steps were applied where appropriate to optimize performance.

The features selected for model training were: `AwardWins`, `awardNominationsExcludeWins`, `numRegion`, `genres`, `TotalReviews`, `decade`, `continent`, `rating_category`, `popularity_category`, `runtime_category`

The features `rating_category`, `popularity_category`, and `runtime_category` were created by binning the continuous variables `rating`, `num_rating`, and `runtimeMinutes`. Remaining numeric features were log-transformed and scaled using MinMax normalization. Categorical features were encoded using MultiLabelBinarizer and one-hot encoding.

The target variable, `titleType`, was refined to include only cinema and television products. Low-frequency categories such as *tvMiniSeries*, *tvSpecial*, and *tvShort* were merged into a single class, *otherTvProducts*, to reduce class imbalance and improve model generalizability

4.2 KNN

The first model applied was the K-Nearest Neighbors (KNN) algorithm, a non-parametric supervised learning method that classifies data points based on the majority class among their k nearest neighbors in the training set. KNN makes no assumptions about the underlying data distribution and is particularly effective in cases where similar instances belong to the same class.

To optimize model performance, we employed GridSearch to determine the best hyperparameters. The optimal configuration was:

- **Distance metric** =cityblock
- **Number of neighbors (k)** = 5
- **Weights strategy** =distance

Given the presence of class imbalance in the dataset, we selected balanced accuracy as the primary scoring metric during hyperparameter tuning. The model achieved an accuracy score of 0.76 on the training set. While accuracy is easy to interpret, it may not reflect true model performance in the presence of class imbalance. For this reason, we also evaluated the model using precision, recall, and the F1-score for a more balanced perspective. As shown in Figure 12, model performance varies across classes, which is

	precision	recall	f1-score	support
movie	0.80	0.86	0.83	1432
otherTvProducts	0.20	0.10	0.13	71
short	0.87	0.88	0.87	570
tvEpisode	0.82	0.91	0.86	1100
tvMovie	0.31	0.28	0.30	196
tvSeries	0.45	0.18	0.26	218
video	0.47	0.34	0.39	178
accuracy			0.77	3765
macro avg	0.56	0.51	0.52	3765
weighted avg	0.74	0.77	0.75	3765

Figure 12: KNN Classification Report

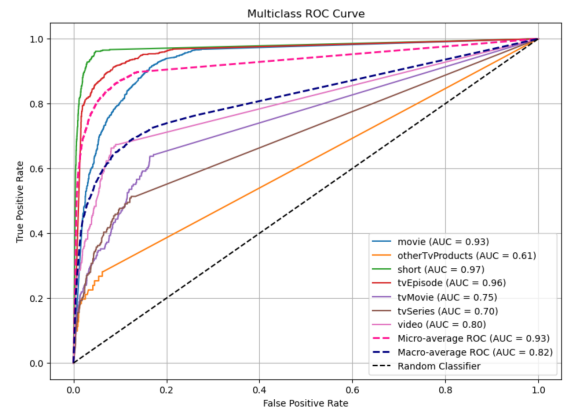


Figure 13: KNN ROC Curve

expected due to the unequal distribution of samples. Despite this, the model achieved a solid accuracy of 0.77 on the test set, indicating good overall performance.

The ROC curves in Figure 13 display, for each class, the trade-off between the True Positive Rate (TPR)

— the proportion of actual positives correctly predicted — and the False Positive Rate (FPR) — the proportion of negatives incorrectly predicted as positives. This visualization is particularly valuable in multiclass classification tasks, as it reveals how well the model distinguishes each class from the rest. On average, the model demonstrated a high TPR-to-FPR ratio of 0.88, suggesting that it is able to reliably identify the correct class in most cases. This indicates that the KNN classifier performs robustly across the diverse categories in the dataset, especially when using the tuned hyperparameters and an appropriate distance metric.

4.3 Naïve Bayes

In this section, we examine the performance of our model using the Naïve Bayes classifier, a probabilistic approach grounded in Bayesian statistics. Naïve Bayes refers to a family of simple algorithms for constructing classifiers—models that assign class labels to instances represented as vectors of feature values, with labels drawn from a finite set of categories.

While there is no single algorithm that defines all Naïve Bayes classifiers, they are unified by a key assumption: that the value of each feature is conditionally independent of the others, given the class label. This simplifying assumption forms the core of the model and enables efficient computation of class probabilities.

	precision	recall	f1-score	support
movie	0.79	0.68	0.73	1432
otherTvProducts	0.10	0.01	0.02	71
short	0.81	0.89	0.85	570
tvEpisode	0.65	0.87	0.75	1100
tvMovie	0.22	0.19	0.21	196
tvSeries	0.25	0.06	0.10	218
video	0.36	0.38	0.37	178
accuracy			0.68	3765
macro avg	0.45	0.44	0.43	3765
weighted avg	0.66	0.68	0.66	3765

Figure 14: Naïve Bayes Classification Report

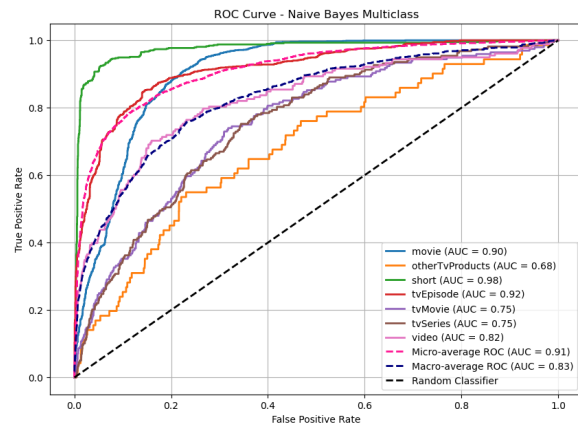


Figure 15: Naïve Bayes ROC Curve

We trained our model using the Multinomial Naïve Bayes algorithm and obtained the evaluation results shown in Figure 14. The model achieved an accuracy of 0.68 on the test set. Among all classes, the "short" category was the easiest to predict, consistent with the results from the KNN classifier, achieving an F1-score of 85%.

Some other classes were not predicted as well; for example, certain categories had F1-scores as low as 0.01 and recall values around 0.02, indicating that the classifier struggled to correctly identify instances from those categories.

The ROC curve shown in Figure 15 illustrates the model's performance across all classes using a one-vs-rest approach. The curves are generally plotted near the top-left corner, suggesting reasonable discriminative power.

In conclusion, while the Naïve Bayes classifier performed adequately, it did not reach the accuracy level obtained with the KNN algorithm.

4.4 Decision Tree

A compelling alternative to probabilistic models like Naïve Bayes and instance-based approaches such as KNN is the Decision Tree classifier. This non-parametric, supervised learning method is valued for its interpretability and ability to model complex decision boundaries without requiring feature scaling. It works by recursively splitting the dataset into increasingly pure subsets based on feature values, ultimately assigning class labels at the leaves. To fine-tune the model and control its complexity, we focused on three critical hyperparameters: `min_samples_split`, `min_samples_leaf`, `max_depth`. Using Randomized Search, we identified the optimal configuration as follows:

- `min_samples_split = 96`
- `min_samples_leaf = 31`
- `max_depth = 12`

The splitting criterion selected was the Gini index, which measures node impurity and helps separate target classes effectively. To improve generalization and reduce overfitting, we applied Cost-Complexity Pruning (CCP), which simplifies the tree by removing branches with little predictive power. The pruning parameter α controls the trade-off between complexity and accuracy: higher values lead to simpler models, while lower values retain complexity but risk overfitting. Our optimal α was 0.00103, indicating a relatively low level of pruning and a balanced model.

On the training set, accuracy improved slightly by 0.02%, from 0.62 to 0.64. Using the best parameters found via Randomized Search and the selected α , the Decision Tree achieved an accuracy of 0.64 on the test set. Figure 18 shows the ROC curve, which is consistently high across genres, indicating good

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.56	0.71	827
1	0.09	0.38	0.14	42
2	0.90	0.86	0.88	377
3	0.86	0.70	0.77	645
4	0.19	0.48	0.27	128
5	0.32	0.58	0.41	132
6	0.40	0.55	0.46	108
accuracy			0.64	2259
macro avg	0.53	0.59	0.52	2259
weighted avg	0.80	0.64	0.69	2259

Figure 16: Decision Tree Classification Report on train set

	precision	recall	f1-score	support
movie	0.93	0.59	0.72	1432
otherTvProducts	0.06	0.31	0.10	71
short	0.92	0.85	0.88	570
tvEpisode	0.88	0.71	0.79	1100
tvMovie	0.17	0.43	0.24	196
tvSeries	0.28	0.50	0.36	218
video	0.37	0.44	0.40	178
accuracy			0.64	3765
macro avg	0.52	0.55	0.50	3765
weighted avg	0.79	0.64	0.69	3765

Figure 17: Decision Tree Classification Report on test set

discrimination between classes. The Precision-Recall curve, more informative for imbalanced datasets, reports a micro-average precision (AP) of 0.72, reflecting strong overall performance, while the macro-average AP of 0.52 indicates weaker performance on minority classes. This difference arises because the ROC curve is less sensitive to class imbalance, calculating TPR and FPR over all instances regardless of class distribution, whereas the Precision-Recall curve provides a more meaningful evaluation in imbalanced scenarios.

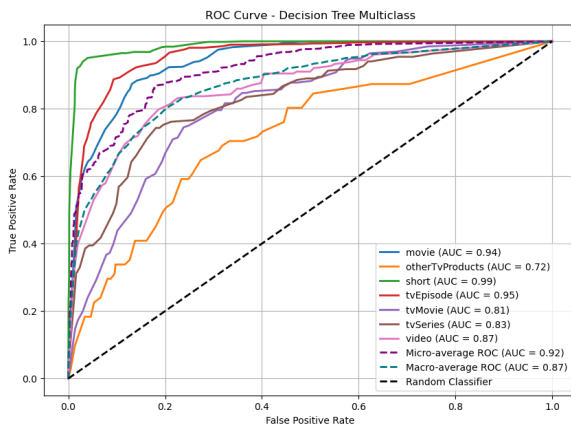


Figure 18: Decision Tree Roc Curve

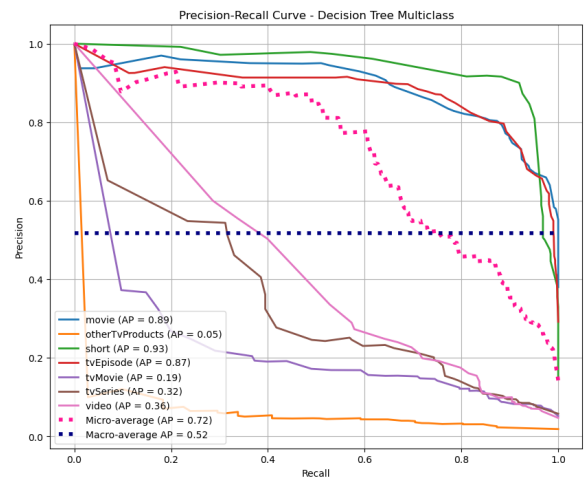


Figure 19: Decision Tree Precision-Recall curve

4.5 Classification conclusion

After evaluating the results of each classification algorithm, we concluded that the K-Nearest Neighbors (KNN) classifier delivered the best performance, achieving an accuracy score of 0.77, followed by the Naïve Bayes classifier with an accuracy of 0.68.

5 Regression

For this task two datasets were used, as for classification: the training and the test datasets resulting from the preprocessing phase. Additional preprocessing was applied to both datasets.

Since regression is the task of predicting a continue numerical feature, all non-numerical and categorical columns were dropped, resulting in a dataset containing the following ones: `num_ratings`, `startYear`, `runtimeMinutes`, `awardWins`, `totalImages`, `totalVideos`, `totalCredits`, `awardNominationsExcludeWins`, `numRegions`, `totalReviews`.

The analysis focused on multivariate regression: at first with just one target variable for which we used a **Linear Regressor** and a **Decision Tree Regressor**, and then multivariate regression with two target variables, using again a **Decision Tree Regressor**. The variables to be used with the **Linear Regressor** were normalized using MinMax scaling and the logarithmic transformation, meanwhile for the **Decision Trees** we kept them as they were to preserve the interpretability of the model. This was done with the second part of the analysis in mind, which is to understand whether the most important features selected by the multivariate-single target **Decision Tree** were useful to better the performance of the **Linear Regressor**.

A dummy regressor was also trained as a tool of comparison, with the `strategy` parameter set to `mean`. The R2 value obtained was of -0.00027 which can be approximated as 0.

5.1 Results of the Multivariate Regression with One Target Variable

The target variable chosen to be predicted in the multivariate -single target regression was `rating`: this is because predicting the rating of a media could potentially mean being able to predict its success.

The Linear Regressor which used all the variables available had the worst performance overall, not even surpassing the dummy classifier. This was probably because of a lack of linear relationships in the data and possibly a number of features which was too high and not informative. In table 7, the results.

Linear Regressor Results		
MSE	MAE	R2
0.44	0.65	-19.7

Table 7: Linear Regressor Results

In image 20, the relationship between the actual rating and the predicted rating, based on the number of `awardWins`: it can be seen how the linear regressor always predicts the same value for all instances.

As for the Decision Tree, the performance was better than the one by both the **Dummy Classifier** and the **Linear Regressor**. The model was trained using a Grid Search using 10 folds and optimizing the following parameters: `criterion`, `min_samples_split`, `min_sample_leaf`, `max_depth` and `ccp_alpha`. The best values were found to be, respectively: `squared_error`, 3, 5, 6 and 0.00.

The results obtained were:

- R2: 0.191
- MSE: 1.558
- MAE: 0.950

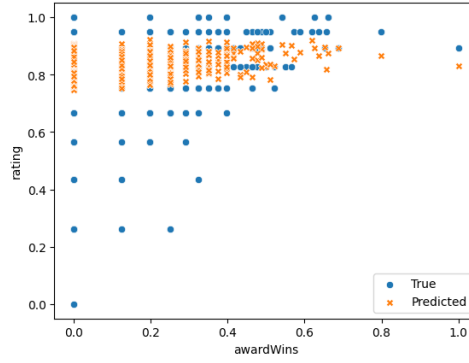


Figure 20: Scatterplot showing behaviour of Linear Regressor

5.2 Feature Importances for the Linear Regressor

To perform this second part of the analysis, the first action made was to print out the feature importances of the **Decision Tree Regressor** (DTR). Many trials were conducted to understand how many of the features found by the DTR were most meaningful for the **Linear Regressor**, finding the following were most helpful: `runtimeMinutes`, `startYear`, `num_ratings`, `totalCredits`, `numRegions`.

Overall, the Linear Regressor performance is still not good, but it did get significantly better, as shown in table 8:

MSE	MAE	R^2
0.0106	0.073	0.0526

Table 8: Linear Regressor with Feature Importance

5.3 Results of the Multivariate Regression with Two Target Variables

This analysis was performed using again a Decision Tree Regressor, which was optimized using a grid search over the same parameter's grid that was used before. This time, the best parameters found were: as for `criterion` it was `absolute_error`, the `ccp_alpha` was 0.01, the `max_depth` was best at None, and the `min_samples_leaf` and `min_samples_split` were respectively at 5 and 3.

As dependent variables `rating` and `num_ratings` were chosen, to continue on the path of predicting the possible popularity of a product. In table 9 the results:

MSE	MAE	R^2
156525714.8	461.503	0.228

Table 9: Decision Tree-Multivariate Regression with 2 Targets

Although the R^2 is at the highest, the MSE and the MAE are also at their highest, signaling that the model might be able to understand some variance in the data but is not capable of predicting the instances properly.

6 Pattern Mining

6.1 Data Preparation

The **data preparation** phase for pattern mining was carried out with the aim of selecting and processing only the most semantically relevant features, in order to extract interpretable patterns. Following an

initial exploratory inspection, variables considered uninformative or redundant for the analysis—such as `originalTitle`, `totalImages`, `totalVideos`, `totalCredits`, and `canHaveEpisodes`—were removed. Additionally, the label `short` from the `titleType` feature was discarded, as this information was already represented in the `genres` attribute.

Categorical feature processing. Continent tagged as `Unknown` were excluded to ensure semantic consistency. The `continent` and `genres` features, being multi-label in nature, were transformed into binary format using the `MultiLabelBinarizer` class, generating a set of binary columns with coherent prefixes (e.g., `genre_Comedy`, `continent_Europe`). The `titleType` feature was instead encoded using `get_dummies`, producing a standard one-hot representation.

Transformation of numerical variables. A mixed approach was adopted for the numerical features, based on discretization techniques guided by empirical distributions and clustering. The `num_ratings` variable, which was highly skewed, was transformed using a logarithmic function (`log1p`) and subsequently standardized via `MinMaxScaler`. It was then discretized using `KMeans` clustering ($k=5$). The resulting clusters were ordered based on the median and renamed according to rating volume (`Very Low` – `Very High`).

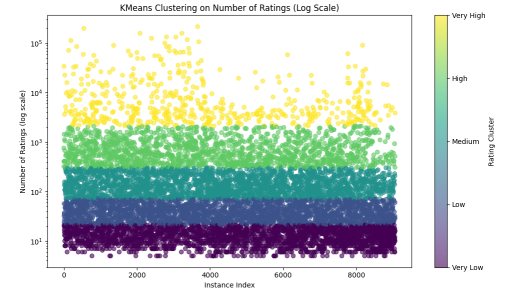


Figure 21: Scatterplot K-Means `num_ratings`

Class	Count
Flop	5022
Considered by Critics	2466
Critically Acclaimed	1580

Table 10: Distribution of the `success_class` variable.

Definition of the `success_class` variable. As the dataset lacked a direct indicator of critical success, the target variable `success_class` was defined through a rule-based approach combining award data and critic reviews. Titles that received at least one award or one nomination were labeled as **Critically Acclaimed**. Those without awards or nominations but with at least one critic review were categorized as **Considered by Critics**. The remaining titles, which had neither reviews nor recognition, were classified as **Flop**.

Logical discretization of additional features. To improve the interpretability of the patterns, additional numerical variables were discretized using manually defined thresholds based on empirical criteria:

- `numRegions`: `Local` if equal to 1, `Limited` if ≤ 5 , `Moderate` if ≤ 10 , and `International` otherwise;
- `rating`: `Poor` (0–5), `Average` (5–6.5), `Good` (6.5–7.5), `Excellent` (>7.5);
- `runtimeMinutes`: discretized into `Short` (≤ 60 min), `Standard` (61–90 min), `Long` (91–120 min), `Extended` (121–150 min), and `Epic` (>150 min) based on fixed duration intervals.

6.2 Frequent Pattern Extraction

The *Apriori* and *FP-Growth* algorithms were employed with varying values of minimum support (`min_support`) and minimum itemset cardinality (`zmin`). The trend in the number of frequent, maximal, and closed itemsets was analyzed as a function of support thresholds, highlighting the exponential growth in the number of patterns with increasing granularity.

6.2.1 Discussion on Frequent Patterns

The analysis was conducted on a transactional dataset obtained by transforming the categorical and binary variables of the original dataset, representing each movie as a distinct set of items.

For each parameter combination, the following were generated:

- **Maximal itemsets** (`target='m'`), which cannot be further extended without reducing support;
- **Closed itemsets** (`target='c'`), which preserve the maximum amount of information while reducing redundancy.

The visualizations obtained reveal that the most frequent patterns are dominated by combinations of film genres, confirming the presence of well-established semantic structures within the dataset.

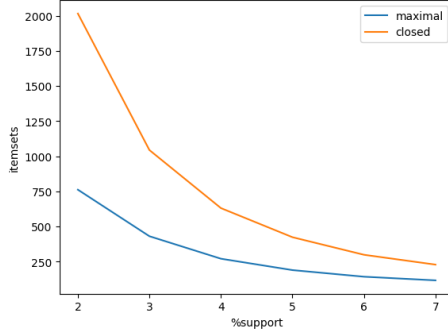


Figure 22: Closed and maximal itemsets for `zmin=2`.

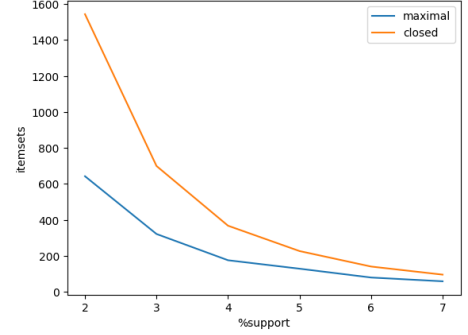


Figure 23: Closed and maximal itemsets for `zmin=3`.

As expected, increasing the support threshold leads to a significant reduction in the number of patterns. Moreover, closed itemsets consistently outnumber maximal ones, as they retain more statistical information. Raising `zmin` from 2 to 3 further decreases the number of patterns, enhancing readability at the cost of simpler pattern loss.

To compare the efficiency and productivity of the Apriori and FP-Growth algorithms, the number of itemsets generated was evaluated at different support levels and itemset sizes. The results are summarized in Table 11.

Table 11: Comparison between Apriori and FP-Growth across support and itemset sizes

Support (%)	Itemset size	Apriori	FP-Growth
8	2	170	113
20	2	15	399
8	3	3	1896
20	3	64	176

As observed, FP-Growth generates a significantly higher number of complex patterns (3-itemsets), particularly at lower support levels. This indicates a greater ability to explore intricate combinations, uncovering latent structures that Apriori tends to miss. In contrast, Apriori proves more efficient in detecting shorter patterns but lacks depth. In light of these findings, FP-Growth with 8% support and `zmin=3` was selected, as it provides an optimal balance between pattern granularity and semantic coverage, demonstrating superiority in both quantitative and qualitative terms for the purposes of this analysis.

6.3 Association Rules Extraction

Association rules were extracted using the FP-Growth algorithm, aimed at generating rules of the form:

$$\text{Antecedent} \Rightarrow \text{Consequent}$$

The rules were evaluated using three main metrics: **Support**, **Confidence** and **Lift**.

The rules listed in Table 12 represent some of the most relevant patterns identified during the analysis. These rules were selected based on particularly high *lift* values (greater than 2.5), indicating a strong and non-random association between antecedent and consequent.

Table 12: Examples of high-lift association rules with predictive potential

Consequent	Antecedent	Conf.	Lift	% Support
rating_cluster_label=Very High N. Ratings	region_distribution=International, type_movie	0.388	6.56	3.14
genre_Mystery	genre_Crime, genre_Drama	0.252	5.10	2.40
success_class=Critically Acclaimed	region_distribution=International, type_movie	0.499	2.86	4.04
genre_Crime	type_tvEpisode, genre_Drama, region_distribution=Local	0.411	3.02	5.26

For instance, the first rule shows that movies distributed internationally and labeled as **type_movie** have a substantially higher likelihood of receiving a very high number of user ratings. This implies a strong relationship between distribution reach and popularity in terms of visibility and audience approval.

Another rule highlights that internationally distributed **type_movie** titles are also frequently labeled as **Critically Acclaimed**, meaning they receive positive recognition from critics. Such rules are especially relevant for predictive tasks involving the qualitative success of movie titles.

Different confidence thresholds were tested to evaluate the trade-off between rule quantity and quality. At $\text{conf} = 30$, 1072 rules were extracted; this dropped to 580 at $\text{conf} = 40$, and just 101 at $\text{conf} = 70$. The threshold of $\text{conf} = 40$ was chosen as the best compromise, offering a diverse yet manageable set of rules with sufficient confidence and frequent lift values above 1.5—indicative of non-trivial associations.

Rule quality was assessed jointly through support, confidence, and lift: high support indicates statistical reliability, confidence reflects predictive strength, and lift captures the true associative relevance. Notably, the rule **success_class=Critically Acclaimed** $\leftarrow$ **region_distribution=International, type_movie** achieved a lift of 2.86 and confidence of 0.50, pointing to a strong link between international distribution and critical success.

Consequent	Antecedent	Conf.	Lift	Support (%)
success_class=Flop	continent_North America, region_distribution=Local	0.689	1.24	21.55
success_class=Flop	rating_category=Average, region_distribution=Local	0.749	1.35	13.52
region_distribution=Local	genre_Drama, continent_North America, success_class=Flop	0.743	1.39	6.15
success_class=Flop	rating_category=Average, continent_North America, region_distribution=Local	0.721	1.30	6.89

Table 13: Examples of association rules with confidence ≥ 0.65 and relevant support

6.3.1 Discussion on Association Rules

Due to the dataset’s class imbalance (see *data preparation*), a relatively low confidence threshold (≥ 0.2) was adopted to retain rules potentially useful for minority classes.

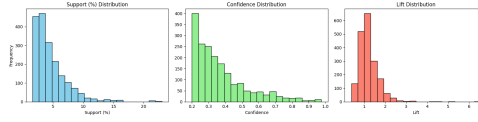


Figure 24: Distribution of association rule metrics: support, confidence, and lift

As shown in Figure 24, most rules exhibit low support (2–6%), consistent with the dataset’s sparsity. Confidence values are typically between 0.2 and 0.5, with few rules exceeding 0.7, indicating limited conditional reliability. Lift values cluster around 1.0, but a significant subset exceeds 1.5—some even beyond 3.0—suggesting the presence of strong and potentially informative co-occurrences.

These results highlight a trade-off: many rules are statistically valid yet moderately reliable. The combination of modest confidence and high lift points to nuanced or partial associations. For downstream use (e.g., in predictive models), rule filtering based on lift and coverage becomes essential to reduce noise.

The subsequent analysis concentrated on rules with consequents involving the target variables `success_class` and `rating_category`, in order to investigate how patterns of critical and audience reception interrelate.

Overall, these distributions confirm that the FP-Growth algorithm was capable of identifying a diverse set of rules, many of which are statistically meaningful, albeit not always highly reliable. The high number of rules with moderate confidence but lift values above 1 suggests the presence of interesting patterns, though sometimes only partially strong or ambiguous. Subsequently, our analysis focused on extracting association rules where the target classes `success_class` and `rating_category` serve as consequents, in order to explore the relationship between critical reception and audience evaluation.

- `success_class` captures the level of critical acclaim, inferred from reviews, awards, and nominations;
- `rating_category` reflects audience appreciation as measured by numerical ratings.

For both target variables, rules with confidence ≥ 0.2 and lift ≥ 1.2 were selected to ensure a meaningful balance between reliability and statistical significance.

For the class `success_class=Flop`, several strong rules emerged:

- `(rating_category=Average, region_distribution=Local)  $\Rightarrow$  success_class=Flop`, with confidence ≈ 0.75 and lift ≈ 1.35 ;
- `(type_tvEpisode, continent_North America, region_distribution=Local)  $\Rightarrow$  success_class=Flop`, with confidence ≈ 0.76 and lift ≈ 1.42 .

Similarly, highly informative rules for `success_class=Critically Acclaimed` include:

- `(region_distribution=International, type_movie)  $\Rightarrow$  success_class=Critically Acclaimed`, with confidence ≈ 0.50 and lift ≈ 2.86 ;
- `(genre_Drama, type_movie)  $\Rightarrow$  success_class=Critically Acclaimed`, with confidence ≈ 0.32 and lift ≈ 1.49 .

Regarding audience perception, predictive rules for `rating_category` were also examined. For the class `rating_category=Good`, coherent co-occurrences emerged, such as:

- `(type_tvEpisode, continent_North America, region_distribution=Local)  $\Rightarrow$  rating_category=Good`, with confidence ≈ 0.55 and lift ≈ 1.57 ;
- `(type_tvEpisode, region_distribution=Local)  $\Rightarrow$  rating_category=Good`, with confidence ≈ 0.48 and lift ≈ 1.53 .

In general, these rules show that certain content segments—such as North American local TV series—tend to be well-received by the public but may simultaneously be overlooked by critics (e.g., rules associated

with **Flop**). This dichotomy between public perception and critical evaluation is further reinforced by rules related to the class **Considered by Critics**, which display lift values above 1.6 in association with internationally distributed films of moderate popularity (**rating_cluster=Medium** or **High N. Ratings**), albeit with lower confidence compared to rules for the **Flop** class.

The strongest rules were subsequently applied to the test set by checking whether the rule *antecedents* were fully contained within the transactions. The outcome was positive: **98% of the transactions** in the test set triggered at least one rule, with an **average of 14.5 applicable rules per transaction**. We analyzed the *top-5* most frequently triggered rules. Some of these, although statistically strong, proved semantically trivial (e.g., **runtime_category=Short** $\leftrightarrow$ **type_tvEpisode**). Therefore, the focus was shifted to rules with informative *consequents*, such as **success_class** and **rating_category**.

Table 14: Some Association rules with **success_class** as antecedent or consequent

Antecedent	Consequent	Confidence	Lift
success_class=Flop	type_tvEpisode	0.461	1.310
success_class=Flop	region_distribution=Local	0.691	1.289
region_distribution=Local	success_class=Flop	0.714	1.289

Table 15: Predictive rules for the **rating_category** class

Antecedent	Consequent	Conf.	Lift
type_tvEpisode, continent_North America, region_distribution=Local	rating_category=Good	0.508	1.634
type_tvEpisode, region_distribution=Local	rating_category=Good	0.492	1.583
type_tvEpisode, region_distribution=Local, success_class=Flop	rating_category=Good	0.479	1.543

This demonstrates how the identified rules are applied to the test set, showing that they are particularly effective for predicting the majority classes such as **Flop** for **success_class** and **Good** for **rating_category**.

6.4 Predictive Use of the Rules

The extracted rules were employed to construct a rule-based classifier, where the prediction for each instance is obtained by selecting, among the applicable rules (i.e., those whose antecedents are contained in the transaction), the one with the highest confidence.

For the variable success_class, the classifier achieved an accuracy of 67.8%, with particularly strong performance on the **Flop** class (precision 0.74, recall 0.93, F1-score 0.82). This result reflects the abundance of high-confidence and high-support rules associated with this category. In contrast, the **Critically Acclaimed** and **Considered by Critics** classes were predicted with low accuracy, especially in terms of recall (0.18 and 0.35 respectively), indicating reduced predictive coverage for these categories.

For the variable rating_category, the overall accuracy reached approximately 49%. Only the **Average** (F1-score 0.61) and **Good** (F1-score 0.51) classes were predicted effectively, while **Excellent** and **Poor** were never predicted (recall 0.00), reflecting the absence of strong rules for minority classes. This outcome underscores the impact of class imbalance and limits the model’s generalizability.

Beyond accuracy, evaluation metrics such as precision, recall, and F1-score confirmed that the rule-based classifier systematically favors majority classes. Its predictive capacity is notably asymmetric, driven by the greater availability of frequent and confident rules for dominant categories.

In summary, while the model performs adequately on prevalent classes, its performance deteriorates sharply for underrepresented ones—highlighting a key limitation of rule-based systems when applied to imbalanced datasets.

6.5 Conclusions

The analysis conducted has demonstrated the effectiveness of pattern mining in extracting interpretable knowledge from complex data within the film domain. Following a thorough phase of data preparation and transformation, the application of both FP-Growth and Apriori algorithms enabled the identification of frequent patterns and association rules characterized by substantial levels of support, confidence, and lift. The FP-Growth algorithm proved preferable due to its ability to generate a significantly higher number of informative rules, particularly for larger itemsets, while maintaining efficient computational performance. The derived rules highlighted meaningful relationships between movie features, geographical distribution, genres, and success metrics (both critical and audience-based). In particular, a clear distinction emerged between patterns associated with critical acclaim and those related to audience ratings, underlining the utility of this approach for comparative and predictive analysis. The rule-based classification system produced mixed results. For the variable `success_class`, the model achieved a notable accuracy of 67.8%, showing strong performance for the majority class `Flop`. However, predictive performance was limited for minority classes. A similar trend was observed for `rating_category`, with an overall accuracy of 49% and poor coverage of the `Excellent` and `Poor` categories. In summary, the rule-based approach demonstrated good descriptive and interpretative potential but exhibited clear limitations in imbalanced supervised learning contexts. The reliance on empirical itemset frequency poses a structural constraint, making it difficult to produce reliable predictions for underrepresented classes.